

架构对比：Qt/C++ 桌面端 vs Vue + Flask Web 端

一、技术栈映射

层次	Qt/C++ 当前方案	Vue + Flask 方案	对应关系
UI 框架	Qt Widgets (QWidget , QLabel , QPushButton)	Vue 3 + Element Plus / Ant Design Vue	1:1 组件映射
布局系统	QHBoxLayout , QVBoxLayout , QGridLayout	CSS Flexbox / Grid + Vue 模板	更灵活
页面路由	QStackedWidget + 手动 setCurrentIndex()	Vue Router (vue-router)	声明式路由，更优雅
状态管理	各 Page 成员变量 + DataRefreshCoordinator 信号槽	Pinia / Vuex 全局状态仓库	集中式，更易维护
动画	QPropertyAnimation , QParallelAnimationGroup	CSS Transition / Animation / Vue <Transition>	CSS 原生更流畅
HTTP 客户端	QNetworkAccessManager (AI 服务调用)	Axios / Fetch API	更简洁
数据库访问	QSqlDatabase + QSqlQuery (直接 SQL)	SQLAlchemy ORM / Flask-SQLAlchemy	ORM 更方便
数据库	SQLite (嵌入式)	SQLite / PostgreSQL / MySQL	可选更多
对话框	QDialog 子类 (9个)	Vue <Modal> 组件 + 表单	复用性更好
表单控件	QLineEdit , QComboBox , QDoubleSpinBox	<el-input> , <el-select> , <el-input-number>	1:1 映射

层次	Qt/C++ 当前方案	Vue + Flask 方案	对应关系
列表/表格	QListWidget , QTableWidget	<el-table> , <el-tree>	功能更强（排序、 分页内置）
系统托盘	QSystemTrayIcon	浏览器不支持（PWA 有 Notification API）	丢失此功能
剪贴板	QClipboard	navigator.clipboard.writeText()	Web API
本地存储	QSettings（注册表/plist）	localStorage / sessionStorage	类似
文件操作	QFile , QFileDialog , QTextStream	<input type="file"> , Blob/Download	受限于浏览器沙箱
构建工具	CMake	Vite / Webpack	前端构建更成熟
语言	C++17	JavaScript/TypeScript + Python	开发效率更高

二、分层架构对比

Qt/C++ 桌面端四层架构

UI 层
MainWindow → QStackedWidget → 12 Pages + 9 Dialogs
23+ Widgets (Sidebar, AiPanel, MetricGrid...)
4 Core Controllers (Shell, Refresh, Backend, AI)
Service 层 – 13 个服务类
CourseService, AiService, AnalyticsService...
DAO 层 – 10 个数据访问类
继承 DaoBase → QSqlDatabase + QSqlQuery
Model 层 – 11 个数据模型
Course, Goal, Role...
数据库 – SQLite (pdp.db)

Vue + Flask 对应架构

前端 (Vue 3 + TypeScript)
App.vue → <router-view> → 12 Pages + 9 Modals
23+ Components (Sidebar, AiPanel, MetricGrid...)
Pinia Store (替代 DataRefreshCoordinator 信号槽)
Axios HTTP Client (替代 QNetworkAccessManager)
API 路由层 (Flask Blueprint)
@app.route("/api/courses") → 14 个蓝图路由
Service 层 (Python) – 13 个服务模块
course_service.py, ai_service.py...
ORM 层 (SQLAlchemy) – 替代手写 DAO
Course.query.all() → 自动生成 SQL
Model 层 (SQLAlchemy Model) – 11 个模型
class Course(db.Model): name, credits...
数据库 – SQLite / PostgreSQL

三、核心模块逐一对比

3.1 主窗口 vs App.vue

Qt/C++	Vue + Flask
MainWindow (766行, QMainWindow)	App.vue (~50行)
QStackedWidget 手动切页	<router-view> 自动匹配路由
QMenuBar + QToolBar 手写	<el-menu> 组件或 Layout 框架
QStatusBar + QProgressBar	全局状态栏组件
QSystemTrayIcon 系统托盘	无 (浏览器环境)

Qt 写法:

```
m_stack->setCurrentIndex(3); // 手动切到第4个页面
```

Vue 写法:

```
<router-link to="/goals">目标</router-link>
<!-- 或 -->
router.push('/goals')
```

3.2 侧边栏导航

Qt/C++	Vue + Flask
SidebarWidget (QFrame)	<Sidebar> 组件
NavigationListWidget (QListWidget)	<el-menu> + <router-link>
QPropertyAnimation 折叠动画	CSS transition: width 0.22s ease-in-out
QParallelAnimationGroup	不需要, CSS 自动同步多个属性
TimeInfoCard / StudentInfoCard	子组件 + Pinia store

Qt 写法 (80 行动画代码):

```
auto* group = new QParallelAnimationGroup(this);
auto* anim1 = new QPropertyAnimation(this, "minimumWidth");
anim1->setDuration(220);
anim1->setEasingCurve(QEasingCurve::InOutQuad);
// ... 设置起止值, 启动动画
```

Vue 写法 (5 行 CSS):

```

<template>
  <aside :class="{ collapsed: isCollapsed }">
    <!-- 导航内容 -->
  </aside>
</template>

<style scoped>
aside {
  width: 260px;
  transition: width 0.22s cubic-bezier(0.4, 0, 0.2, 1);
}
aside.collapsed { width: 56px; }
</style>

```

3.3 CRUD 页面（以课程页为例）

Qt/C++	Vue + Flask
CoursesPage (BasePage 子类)	CoursesPage.vue
QTableWidget 手动填充行	<el-table :data="courses"> 数据绑定
QLineEdit 搜索 → textChanged 信号 → refresh()	v-model 双向绑定 + computed 过滤
CourseEditorDialog (QDialog, ~200行)	<CourseModal> (~50行模板)
QPushButton + connect() 信号槽	@click="handleAdd" 事件绑定
CourseService::create() 静态方法	courseApi.create() Axios 调用

Qt 写法（手动填充表格）：

```

void CoursesPage::refresh() {
  QList<Course> courses = CourseService::getAll();
  m_table->setRowCount(courses.size());
  for (int i = 0; i < courses.size(); ++i) {
    m_table->setItem(i, 0, new QTableWidgetItem(courses[i].name));
    m_table->setItem(i, 1, new QTableWidgetItem(courses[i].code));
    // ... 逐列填充
  }
}

```

Vue 写法（声明式数据绑定）：

```
<template>
  <el-table :data="filteredCourses">
    <el-table-column prop="name" label="课程名称" />
    <el-table-column prop="code" label="课程代码" />
    <el-table-column prop="credits" label="学分" />
    <el-table-column prop="score" label="成绩" />
  </el-table>
</template>

<script setup>
import { computed } from 'vue'
import { useCourseStore } from '@stores/course'

const store = useCourseStore()
store.fetchAll()

const filteredCourses = computed(() =>
  store.courses.filter(c => c.name.includes(search.value))
)
</script>
```

3.4 数据刷新协调器 vs Pinia Store

Qt/C++	Vue + Flask
DataRefreshCoordinator (手动信号槽连接)	Pinia Store (自动响应式)
emit dataChanged(DataDomain::Goals)	store.updateGoal() → 所有订阅组件自动更新
手动维护 12 个页面的刷新映射	Vue 的响应式系统自动追踪依赖
QMap<int, std::function> 注册表	不需要，Vue Proxy 自动处理

Qt 写法（手动协调）：

```
void DataRefreshCoordinator::refreshByDomain(DataDomain domain) {
    m_overview->refresh(); // 总是刷新
    m_timeline->refresh();
    switch (domain) {
        case DataDomain::Goals:
            m_goals->refresh();
            m_analysis->refresh();
            m_resume->refresh();
            break;
        // 每个域都要手动列出需要刷新的页面...
    }
}
```

Vue 写法（自动响应式）：

```
// stores/goal.js
export const useGoalStore = defineStore('goal', {
  state: () => ({ goals: [], stats: {} }),
  actions: {
    async fetchAll() {
      this.goals = await goalApi.getAll()
      this.stats = await goalApi.getStatistics()
    },
    async create(goal) {
      await goalApi.create(goal)
      await this.fetchAll() // 更新 store → 所有使用它的组件自动刷新
    }
  }
})

// 任何页面中：
const goalStore = useGoalStore()
// goalStore.goals 变化时，所有绑定它的 <template> 自动重新渲染
```

3.5 AI 服务

Qt/C++	Vue + Flask
AiService 静态类 (C++)	ai_service.py (Flask 路由) + aiApi.js (前端调用)
QNetworkAccessManager 同步调用	axios.get('/api/ai/health') 异步

Qt/C++	Vue + Flask
QEventLoop 阻塞等响应（有瑕疵）	async/await 天然异步，不阻塞 UI
AiContextMediator eventFilter 划词	@mouseup 事件 + window.getSelection()
AiPanelWidget 折叠面板	Vue <Transition> + CSS 动画

Qt 写法（同步阻塞，有 UI 卡顿风险）：

```
QEventLoop loop;
QNetworkReply* reply = manager.get(request);
QTimer::singleShot(2000, &loop, &QEventLoop::quit);
connect(reply, &QNetworkReply::finished, &loop, &QEventLoop::quit);
loop.exec(); // 阻塞主线程！
```

Vue 写法（天然异步，零阻塞）：

```
async function checkAiHealth() {
  try {
    const res = await axios.get('/api/ai/health', { timeout: 2000 })
    aiStore.status = res.data.model_loaded ? 'remote' : 'loading'
  } catch {
    aiStore.status = 'local' // 自动降级
  }
}
```

3.6 事件过滤器 vs DOM 事件

Qt/C++	Vue + Flask
AiContextMediator::eventFilter() 全局拦截	document.addEventListener('mouseup', ...)
qobject_cast<QLabel*> 逐类型检测	window.getSelection().toString() 一行搞定
manhattanLength() 判断拖拽距离	不需要，浏览器原生支持文本选中
QEvent::ChildAdded 递归设置可选	不需要，DOM 元素默认文本可选
QTimer::singleShot(50, ...) 延迟检测	不需要，mouseup 时选区已确定

Qt 写法（~150 行 eventFilter 代码）：

```
bool AiContextMediator::eventFilter(QObject* watched, QEvent* event) {
    if (event->type() == QEvent::ChildAdded) { makeTextSelectable(...); }
    if (event->type() == QEvent::MouseButtonPress) { ... }
    if (event->type() == QEvent::MouseMove) { ... }
    if (event->type() == QEvent::MouseButtonRelease) {
        QTimer::singleShot(50, this, [this]() {
            QString text = selectedTextFromWidget(QApplication::focusWidget());
            if (!text.isEmpty()) showFloatingMenu(QCursor::pos(), text);
        });
    }
}
```

Vue 写法 (~10 行):

```
document.addEventListener('mouseup', (e) => {
    const text = window.getSelection().toString().trim()
    if (text.length > 1) {
        showFloatingMenu(e.clientX, e.clientY, text)
    }
})
```

3.7 数据库层

Qt/C++	Vue + Flask
DaoBase 基类 + QSqlDatabase	SQLAlchemy db.Model 基类
手写 QSqlQuery::prepare/bindValue/exec	Course.query.filter_by(id=1).first()
mapFromQuery() 手动映射字段	ORM 自动映射
10 个 DAO 类，每个 ~100 行	10 个 Model 类，每个 ~20 行
schema.sql 手动建表	db.create_all() 自动迁移

Qt 写法 (手写 SQL, ~100 行/DAO):

```
QList<Course> getAll() const {
    QList<Course> courses;
    QSqlQuery query(m_db);
    query.exec("SELECT * FROM courses ORDER BY updated_at DESC");
    while (query.next()) courses.append(mapFromQuery(query));
    return courses;
}

Course mapFromQuery(const QSqlQuery& q) const {
    Course c;
    c.id = q.value("id").toInt();
    c.name = q.value("name").toString();
    c.credits = q.value("credits").toDouble();
    // ... 12 个字段逐个映射
    return c;
}
```

Flask + SQLAlchemy 写法 (~5 行):

```
class Course(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(200), nullable=False)
    code = db.Column(db.String(50))
    credits = db.Column(db.Float, default=0)
    semester = db.Column(db.String(20))

# 查询：一行搞定
courses = Course.query.order_by(Course.updated_at.desc()).all()
```

四、优劣对比总结

维度	Qt/C++ 桌面端	Vue + Flask Web 端
开发效率	低（手写布局、SQL、信号槽）	高（声明式模板、ORM、响应式）
运行性能	高（原生编译，无虚拟机）	中（浏览器 JS 引擎）
UI 表现力	中（Widgets 原生风格）	高（CSS 无限可能）
跨平台	好（编译一次各平台运行）	最好（浏览器即平台）

维度	Qt/C++ 桌面端	Vue + Flask Web 端
离线使用	好（纯本地）	差（依赖网络，PWA 部分支持）
部署	需安装	浏览器打开即用
系统集成	强（托盘、文件系统、注册表）	弱（浏览器沙箱限制）
代码量	~15,200 行	预估 ~6,000-8,000 行
学习曲线	高（C++ + Qt 特有机制）	中（JS/Python 生态更友好）
维护成本	高（C++ 编译慢、调试难）	低（热更新、浏览器 DevTools）

一句话总结

Qt/C++ 桌面端的优势在于**性能和系统集成**，代价是开发效率低、代码量大；Vue + Flask 的优势在于**开发效率和 UI 表现力**，代价是依赖网络和浏览器环境。同一个系统如果用 Vue + Flask 重写，代码量预计减少 40-50%，但会丢失系统托盘、本地文件直接访问等桌面端独有能力。